DSAA3073: Theories in Data Science

Week 4: Bias-Variance Tradeoff & Regularization

Cognitive Journey Learning Sheet (v4)

Wei Wang · DSA, HKUST(GZ) · 2026-07-08

Opening Challenge: The Model Selec

This phenomenon—where complex models fit training data perfectly but fail on new data—was recognized as early as the 1960s but only formalized mathematically in the 1970s with the bias-variance decomposition.

🎯 Opening Challenge

The Housing Price Predictor

You're building a model to predict housing prices. You have three options:

1. **Simple:** price $= \beta_0 + \beta_1 \cdot \text{sqft}$
2. **Medium:** price $= \beta_0 + \beta_1 \cdot \text{sqft} + \beta_2 \cdot \text{bedrooms} + \beta_3 \cdot \text{location} + \cdots$
3. **Complex:** 100 features including polynomials, interactions, neighborhood statistics

Observations:

- Model 1: Training error = 15%, Test error = 16%
- Model 2: Training error = 8%, Test error = 9%
- Model 3: Training error = 2%, Test error = 25%

Question: Which model should you deploy? Why does the most complex model perform worst on test data despite fitting training data best?

The answer lies in understanding the fundamental tradeoff between bias (systematic error) and variance (estimation instability). This week, we'll formalize this tradeoff and develop regularization as a principled solution.

✓ Checkpoint

Before we begin: In the opening challenge, Model 3 had 2% training error but 25% test error. What does this 23-percentage-point gap tell you about the model's behavior?

Answer: The large gap indicates the model is memorizing training data rather than learning generalizable patterns—a hallmark of overfitting.

The Bias-Variance Decomposition

Motivation: We've seen that complex models can fail spectacularly on test data despite perfect training performance. But why? The bias-variance decomposition provides a mathematical microscope to examine prediction error, revealing that it stems from three distinct sources: systematic model limitations (bias), sensitivity to random sampling (variance), and irreducible noise. Understanding this decomposition is the key to controlling model complexity.

← From Week 2: VC Theory

We learned that larger hypothesis classes (higher VC dimension) can fit more patterns but require more data to generalize. The bias-variance decomposition quantifies this tradeoff numerically: large classes have low bias but high variance.

The Diagnostic Puzzle: Why Isn't MSE Enough?

? Student Question

"We already have MSE to measure prediction error. It's just one number—simple and clear. Why do we need to decompose it into three components? Isn't that just making things more complicated?"

This is a reasonable question. Let's investigate with a concrete scenario.

Setup and Motivation

Consider the **regression problem** (the task of predicting a continuous output variable from input features): we observe (x_i, y_i) where $y_i = f^*(x_i) + \varepsilon_i$ with $\mathbb{E}(\varepsilon_i) = 0$ and $\mathrm{Var}(\varepsilon_i) = \sigma^2$.

Here, $f^*(x)$ is the **true underlying function** (the ideal relationship between inputs and outputs without noise), ε_i is the **observation noise** (random error in measurements), and $\hat{f}_n(x)$ is our **estimator** (the function we learn from n training samples).

Definition: Mean Squared Error

At a fixed input x, the **prediction error** (the expected squared difference between our prediction and the true value) is measured by the **Mean Squared Error** (MSE):

$$\mathrm{MSE}(x) = \mathbb{E}\Big(\big(y - \hat{f}_n(x)\big)^2\Big) \tag{1}$$

where the expectation is over the training sample used to fit $\hat{f}_n$ and the fresh test noise in y. This expectation accounts for randomness from two sources: (1) which particular training samples we happened to observe, and (2) the noise in the new test observation y.

Example: Housing Prices - Setup

Throughout this section, imagine predicting house prices from square footage. We have $n = 50$ training houses, and the true relationship is roughly quadratic: $f^*(x) = 100 + 0.5x + 0.0001x^2$ (in thousands of dollars), but our observations have noise $\varepsilon_i \sim \mathcal{N}(0, 20^2)$.

The Fragmentation Problem: Three Opaque Numbers

Suppose we train two models on the same housing data and evaluate them at $x = 2000$ sq ft:

Model A (Simple Linear):

Model B (High-degree Polynomial):

⚠ **Crisis**

The Diagnostic Gap

Both models have identical MSE = 0.36. But which one should we use?

- If we collect more training data (n = 50 → 100), which model will improve more?
- If the noise level decreases ($\sigma^2 = 400 \to 100$), which benefits more?
- If we need to explain the model to stakeholders, which is more trustworthy?

MSE alone cannot answer these questions!

Without decomposition, we're forced to treat MSE as a black box:

Question	Information Needed	Available from MSE?	Consequence
Which improves with more data?	Variance magnitude	✗ No	Trial and error
Is model class appropriate?	Bias magnitude	✗ No	Blind to systematic error
How much room for improvement?	Irreducible vs reducible	✗ No	Can't gauge ceiling
Why is test error high?	Source of error	✗ No	Can't diagnose root cause

The fundamental issue: MSE aggregates THREE conceptually different error sources into one number. To control test error, we need to see them separately.

Revealing the Three Sources

Let's peek inside the MSE = 0.36 for both models:

Model	Bias²	Variance	Noise	Interpretation
A (Linear)	0.25	0.02	0.09	Wrong model class (can't capture quadratic)

B (Degree-10)	0.02	0.25	0.09	Right flexibility, but fits noise

Now the questions have answers:

Q: Which improves with more data (n = 50 → 100)?

Model	At n=50	At n=100	Improvement	Why?
A (Linear)	0.36	0.34	Minimal (↓0.02)	Bias dominates, can't reduce
B (Degree-10)	0.36	0.14	**Large (↓0.22)**	Variance halves with 2× data

Answer: Model B improves dramatically (0.36 → 0.14) because its error is variance-dominated. Model A barely improves (0.36 → 0.34) because its error is bias-dominated—more data can't fix the wrong model class!

Q: Which is more trustworthy for explanation?

Q: How much can we improve if we tune λ or change architecture?

💡 **Aha Moment**

The Unification: Three Independent Sources

MSE decomposes into THREE independent components:

$$\text{MSE} = \underbrace{\text{Bias}^2}_{\text{Systematic error}} + \underbrace{\text{Variance}}_{\text{Stochastic error}} + \underbrace{\sigma^2}_{\text{Irreducible}} \tag{2}$$

Each component:

- Comes from a DIFFERENT domain (model choice, sampling, measurement)
- Has DIFFERENT remedies (change model, collect data, better sensors)
- Behaves DIFFERENTLY as n increases (bias stable, variance ↓, noise stable)

Knowing the decomposition transforms diagnosis from guesswork to science.

Domain Table: Understanding the Three Sources

Component	Domain	What It Measures	How to Reduce	Behavior as n→∞
Bias²	Model choice (systematic)	Mismatch between model class and truth	Use more flexible model class	Stays constant
Variance	Sampling (stochastic)	Sensitivity to which training samples we got	More data or regularization	Decreases O(1/n)
Noise σ^2	Measurement (irreducible)	Inherent randomness in observations	Better sensors/ equipment	Stays constant

Key insight: Bias and variance arise from DIFFERENT sources, so they require DIFFERENT solutions. MSE hides this distinction.

Before/After Workflow

Before decomposition (black box trial-and-error):

1. Train model → MSE = 0.82
2. "That's bad. Try… regularization?"
3. Add ridge with λ=0.1 → MSE = 0.65
4. "Better! Try stronger λ=1?"
5. λ=1 → MSE = 0.78
6. "Worse! Go back to λ=0.1?"
7. (Repeat random search…)

After decomposition (targeted diagnosis):

1. Train model → MSE = 0.82
2. **Decompose:** Bias² = 0.05, Variance = 0.68, Noise = 0.09
3. **Diagnose:** Variance-dominated problem (0.68 >> 0.05)
4. **Prescribe:** Need regularization or more data (not more flexibility!)
5. Add ridge with λ=1 → MSE = 0.18
6. **Verify:** New decomposition: Bias² = 0.04, Variance = 0.05
   - Successfully traded tiny bias increase for massive variance reduction
7. Done!

The transformation: "Try random things" → "Diagnose, prescribe, verify"

The Fundamental Decomposition

Now that we understand WHY we need the decomposition, let's see the formal statement.

Theorem: Bias-Variance Decomposition

At a fixed x, assume $y = f^*(x) + \varepsilon$ with $\mathbb{E}(\varepsilon \mid x) = 0$, $\mathrm{Var}(\varepsilon \mid x) = \sigma^2$, and the fresh test noise ε independent of the training sample S. For any estimator $\hat{f}_n$ trained on S, the MSE (expectation over training samples S and fresh test noise ε) decomposes as:

$$\mathrm{MSE}(x) = \underbrace{\left(\mathbb{E}_S\left(\hat{f}_n(x)\right) - f^*(x)\right)^2}_{\text{Bias}^2} + \underbrace{\mathrm{Var}_S\left(\hat{f}_n(x)\right)}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible}} \tag{3}$$

The three terms represent distinct error sources:

- **Bias squared** $\left(\mathbb{E}_S\left(\hat{f}_n(x)\right) - f^*(x)\right)^2$: systematic error from model limitations (e.g., fitting a line to curved data)
- **Variance** $\mathrm{Var}_S\left(\hat{f}_n(x)\right)$: prediction instability due to random training sample variation
- **Irreducible error** σ^2: inherent noise in the data that no model can eliminate

Proof. **Stage 1: Isolate irreducible error**

Let $y = f^*(x) + \varepsilon$ where $\mathbb{E}(\varepsilon) = 0$. We decompose:

$$\begin{aligned}\mathrm{MSE}(x) &= \mathbb{E}\Big(\big(y - \hat{f}_n(x)\big)^2\Big) \\ &= \mathbb{E}\Big(\big(f^*(x) + \varepsilon - \hat{f}_n(x)\big)^2\Big) \\ &= \mathbb{E}\Big(\big(f^*(x) - \hat{f}_n(x)\big)^2\Big) + \mathbb{E}(\varepsilon^2) + 2\mathbb{E}\Big(\varepsilon\big(f^*(x) - \hat{f}_n(x)\big)\Big)\end{aligned} \tag{4}$$

The cross term vanishes since $\mathbb{E}(\varepsilon) = 0$ and ε is independent of $\hat{f}_n$. Thus:

$$\mathrm{MSE}(x) = \mathbb{E}\Big(\big(f^*(x) - \hat{f}_n(x)\big)^2\Big) + \sigma^2 \tag{5}$$

Stage 2: Separate bias and variance

Now add and subtract $\mathbb{E}\big(\hat{f}_n(x)\big)$:

$$\begin{aligned}\mathbb{E}\Big(\big(f^*(x) - \hat{f}_n(x)\big)^2\Big) &= \mathbb{E}\Big(\big(f^*(x) - \mathbb{E}\big(\hat{f}_n(x)\big) + \mathbb{E}\big(\hat{f}_n(x)\big) - \hat{f}_n(x)\big)^2\Big) \\ &= \big(f^*(x) - \mathbb{E}\big(\hat{f}_n(x)\big)\big)^2 + \mathbb{E}\Big(\big(\hat{f}_n(x) - \mathbb{E}\big(\hat{f}_n(x)\big)\big)^2\Big) \\ &\quad + 2\big(f^*(x) - \mathbb{E}\big(\hat{f}_n(x)\big)\big)\mathbb{E}\big(\hat{f}_n(x) - \mathbb{E}\big(\hat{f}_n(x)\big)\big)\end{aligned} \tag{6}$$

The last term is zero since $\mathbb{E}\big(\hat{f}_n(x) - \mathbb{E}\big(\hat{f}_n(x)\big)\big) = 0$. Therefore:

$$\mathrm{MSE}(x) = \big[\mathbb{E}\big(\hat{f}_n(x)\big) - f^*(x)\big]^2 + \mathrm{Var}\big[\hat{f}_n(x)\big] + \sigma^2 \tag{7}$$

■

🎯 Key Takeaway

The bias-variance decomposition reveals three sources of prediction error. Operationally, imagine repeatedly resampling the training set and refitting the model: bias measures systematic displacement from $f^*(x)$, while variance measures how much predictions move across refits.

1. **Bias:** Systematic error from model assumptions (e.g., fitting a line to nonlinear data)
2. **Variance:** Error from sensitivity to training data fluctuations
3. **Irreducible error:** Noise in the data that no model can eliminate

The Bias-Variance Tradeoff

Why do bias and variance move in opposite directions?

🧠 **Mental Model: The Bias-Variance Seesaw**

```
Simple Model          Optimal          Complex Model
├──────────────────────┼──────────────────────┤
High Bias             Sweet              High Variance
Low Variance          Spot               Low Bias
Underfits                                Overfits

Total Error = Bias² + Variance + σ²
```

Think of model selection as balancing on a seesaw: push toward complexity (right) and variance rises while bias falls; pull toward simplicity (left) and bias rises while variance falls. The optimal model is where total error is minimized—rarely at either extreme.

Why the tradeoff?

With fixed sample size n, we face a fundamental constraint:

1. **Simple models (high bias, low variance):**
   - Can't capture complex patterns (high bias)
   - But few parameters → stable estimates (low variance)
   - Example: Linear model for quadratic truth
2. **Complex models (low bias, high variance):**
   - Can capture complex patterns (low bias)
   - But many parameters → unstable estimates (high variance)
   - Example: Degree-15 polynomial for quadratic truth
3. **Just-right models (balanced):**
   - Flexibility matches truth (moderate bias)
   - Parameters controlled by data (moderate variance)
   - Example: Degree-2 polynomial for quadratic truth

Concrete Example: Polynomial Regression

Let's make this concrete with polynomial fitting.

Example: Polynomial Fitting

Consider fitting polynomials of degree d to data from $f^*(x) = \sin$ noise $\sigma = 0.3$ and $n = 50$ samples.

The values below are illustrative of the typical qualitative pattern

Degree 1 (Linear):

- Bias²: 0.25 (cannot capture sinusoidal oscillations)
- Variance: 0.02 (only 2 parameters to estimate)
- Noise: 0.09
- MSE = 0.25 + 0.02 + 0.09 = **0.36**
- **Issue:** Underfitting—model class too restrictive

Degree 5:

- Bias²: 0.02 (flexible enough to approximate sine well)
- Variance: 0.08 (6 parameters, manageable with n=50)
- Noise: 0.09
- MSE = 0.02 + 0.08 + 0.09 = **0.19** ← **Optimal**
- **Sweet spot:** Good approximation without overfitting

Degree 15:

- Bias²: 0.01 (can fit almost anything)
- Variance: 0.35 (16 parameters, severe overfitting with n=50)
- Noise: 0.09
- MSE = 0.01 + 0.35 + 0.09 = **0.45**
- **Issue:** Overfitting—fitting noise instead of signal

The "sweet spot" depends on sample size n, noise level σ^2, and the smoothness of f^*. With more data, we can often afford higher complexity, but there is no universal law such as one fixed scaling for the optimal degree.

Degree	Bias²	Variance	Noise	MSE	Verdict
1	0.25	0.02	0.09	0.36	High bias (underfit)
3	0.08	0.05	0.09	0.22	Good balance
5	0.02	0.08	0.09	0.19	Optimal
10	0.01	0.25	0.09	0.35	High variance (overfit)
15	0.01	0.35	0.09	0.45	Severe overfit

The U-shaped curve:

As degree increases:

← From Week 3: Rademacher Complexity

We proved that generalization error is bounded by $O\left(\sqrt{\frac{\log|\mathcal{H}|}{n}}\right)$ for finite hypothesis classes. Polynomial degree d directly controls $|\mathcal{H}|$—higher degree means larger class, slower convergence, and higher variance for fixed n. The optimal degree balances approximation (bias) and estimation (variance).

How Sample Size Affects the Tradeoff

🤔 **Pause and Think**

Question: If we double the sample size from $n = 50$ to $n = 100$, how might the optimal polynomial degree change?

Hint: Variance decreases with more data, so a somewhat richer model may become viable.

Answer: It may increase, but the amount depends on the unknown signal smoothness, noise level, sampling design, and criterion used. The qualitative lesson is the bias-variance tradeoff, not a universal formula for d^*.

Sample size scaling:

n	Optimal Degree	Bias² (at optimal)	Variance (at optimal)	Why?
20	3	0.08	0.12	Small n → high variance → need simplicity
50	5	0.02	0.08	Moderate n → can afford some complexity
100	7	0.01	0.05	Large n → variance controlled → more flexibility
500	10	0.005	0.02	Very large n → variance low → high flexibility OK

The pattern: As n increases, variance decreases for any fixed model, allowing us to move toward more complex models without catastrophic overfitting.

Practical Implications

🎯 Key Takeaway

Diagnostic workflow using bias-variance decomposition:

1. **Compute decomposition** (via bootstrap or cross-validation):
   - Bias^2 = systematic error
   - Variance = prediction instability
   - Noise = irreducible (estimate from replicate measurements if available)
2. **Diagnose dominant source:**
   - If Bias^2 >> Variance: Model too simple → increase flexibility
   - If Variance >> Bias^2: Model too complex → add regularization or more data
   - If both moderate: Near optimal
3. **Prescribe remedy:**
   - High bias: More features, higher polynomial degree, deeper network
   - High variance: Regularization (ridge, lasso), more data, ensemble methods
   - High noise: Better measurements, feature engineering
4. **Verify:**
   - Recompute decomposition after intervention
   - Confirm you reduced the dominant component

📊 Confidence Check

Self-Assessment: Can you...

1. Derive the bias-variance decomposition from scratch, explaining each step?
2. Given a specific estimator and data distribution, compute bias and variance?
3. Explain why bias and variance typically move in opposite directions as model complexity changes?
4. For a given error decomposition, prescribe the appropriate remedy?

If unsure on any, revisit the theorem and proof before continuing.

🔨 Construct Your Own

Design Your Own Example

Create a regression scenario where:

1. Choose a true function $f^*(x)$ (can be piecewise, trigonometric, polynomial)
2. Pick two model classes: one too simple (high bias), one too complex (high variance)
3. For each model, sketch qualitatively how bias and variance change with sample size n
4. Identify the "crossover" point where the complex model becomes preferable

Extension: How does noise level σ^2 affect this crossover?

Progress on backbone: The bias-variance decomposition reveals that prediction error splits into three independent components from different domains. We can't eliminate irreducible noise,

but we **can** control the bias-variance tradeoff. The question becomes: how do we systematically reduce variance without accepting unlimited bias?

We've seen that controlling variance is crucial, but **why** does variance become problematic in the first place? The answer lies in a deep mathematical insight: when we fit complex models to noisy data, we're solving an **inverse problem**—and not all such problems are well-behaved. Understanding this connection transforms model selection from an art into a science.

Ill-Posed Inverse Problems

Motivation: We've seen that controlling variance is crucial. But **why** does variance explode in the first place? The answer lies in understanding that many learning problems are **inverse problems**—we observe effects (outputs) and must infer causes (parameters). When this inverse problem is "ill-posed," small measurement noise gets catastrophically amplified. Understanding ill-posedness explains why regularization isn't optional—it's fundamental.

The Direct Approach: Just Invert the Matrix

For linear regression, the solution seems straightforward:

$$y = X\beta + \varepsilon \implies X^\top y = X^\top X\beta \implies \beta = (X^\top X)^{-1} X^\top y \tag{8}$$

"Just invert $X^\top X$ and we're done!"

Let's check if this works.

Hadamard's Well-Posedness Criteria

Definition: Well-Posed Problem (Hadamard, 1902)

A problem is **well-posed** if it satisfies three criteria:

1. **Existence:** A solution exists for all admissible inputs
2. **Uniqueness:** The solution is unique
3. **Stability:** Small changes in input lead to small changes in output

A problem failing any criterion is **ill-posed**.

For ordinary least squares (OLS, defined formally in Section 3.1) with $X \in \mathbb{R}^{n \times d}$ where $n > d$:

The third criterion is where things go wrong.

The Structural Obstacle: Small Eigenvalues

Consider what happens when $X^\top X$ has small eigenvalues.

Example: Near-Collinearity Crisis

Suppose $X^\top X$ has eigenvalue decomposition with eigenvalues:

$$\mu_1 = 100, \quad \mu_2 = 0.001 \tag{9}$$

The **condition number** is:

$$\kappa = \frac{\mu_{\max}}{\mu_{\min}} = \frac{100}{0.001} = 100000 \tag{10}$$

What this means: The problem loses approximately $\log_{10}(100000) = 5$ digits of precision.

⚠ **Crisis**

The Instability Crisis: Invertible $\neq$ Stable

Even when $X^\top X$ is invertible, small eigenvalues cause catastrophic instability.

With condition number $\varkappa = 100{,}000$:

- Tiny measurement error (0.001 in y) $\rightarrow$ huge parameter change (52$\times$ in β)
- 5 digits of precision lost in the solution
- Solution exists and is unique, but **unusable in practice**

Numerical Demonstration

Let's see this concretely. Suppose:

$$X^\top X = \begin{pmatrix} 100 & 0 \\ 0 & 0.001 \end{pmatrix}, \quad f^* = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \tag{11}$$

Case 1: Clean observation

$$y = \begin{pmatrix} 10.000 \\ 5.000 \end{pmatrix} \Longrightarrow \hat{\beta} = (X^\top X)^{-1} y = \begin{pmatrix} 1.00 \\ 2.00 \end{pmatrix} \tag{12}$$

Perfect! Recovered the true parameters.

Case 2: Tiny perturbation (0.001 change in second component)

$$y = \begin{pmatrix} 10.001 \\ 5.000 \end{pmatrix} \Longrightarrow \hat{\beta} = (X^\top X)^{-1} y = \begin{pmatrix} 1.00 \\ 52.00 \end{pmatrix} \tag{13}$$

The catastrophe: 0.1% noise in input $\rightarrow$ 2500% error in output!

Scenario	Input Noise	Output $\hat{\beta}_2$	Error	Relative Error
Clean	0	2.00	0	0%
Tiny noise	0.001	52.00	50.00	2500%
Small noise	0.01	502.00	500.00	25000%

Why this happens:

The inverse $(X^\top X)^{-1}$ has eigenvalues $\frac{1}{\mu_j}$:

$$\frac{1}{\mu_1} = \frac{1}{100} = 0.01, \quad \frac{1}{\mu_2} = \frac{1}{0.001} = 1000 \tag{14}$$

Small eigenvalues become **large divisors** in the inverse, amplifying noise by factors up to 1000$\times$!

The Fundamental Issue

🎯 **Key Takeaway**

Invertibility is necessary but not sufficient for stability.

Three levels of matrix conditioning:

1. **Singular** (det = 0): No unique solution
2. **Ill-conditioned** (small but nonzero eigenvalues): Unique but unstable solution
3. **Well-conditioned** (all eigenvalues $\geq c > 0$): Stable solution

OLS addresses level 1 (invertibility) but not level 2 (stability).

Condition Number	Digits Lost	Amplification	Category	Usable?
< 10	< 1	$< 10\times$	Well-conditioned	✓ Yes
10-100	1-2	10-100×	Moderate	✓ Carefully
100-10,000	2-4	100-10K×	Ill-conditioned	✗ Dangerous
$> 10{,}000$	> 4	> 10K×	Severely ill-posed	✗ Unusable

In our housing example from earlier, $\varkappa \approx 18{,}750 \rightarrow$ category "severely ill-posed."

The Transformation: Regularization as Stabilization

How do we transform an ill-posed problem into a well-posed one?

💡 **Aha Moment**

The Resolution: Bound Eigenvalues from Below

Add λI to $X^\top X$, where $\lambda > 0$ is a **regularization parameter** that controls stability:

$$(X^\top X + \lambda I)^{-1} \tag{15}$$

Effect on eigenvalues:

- Original: $\mu_1 = 100, \mu_2 = 0.001$
- Modified: $\mu_1 + \lambda = 110, \mu_2 + \lambda = 10.001$ (with $\lambda = 10$)

Effect on condition number:

- Original: $\kappa = \frac{100}{0.001} = 100000$
- Modified: $\kappa' = \frac{110}{10.001} \approx 11$

Transformation: Severely ill-posed → well-conditioned!

Why this works:

The modified inverse has eigenvalues:

$$\frac{1}{\mu_j + \lambda} \tag{16}$$

By bounding eigenvalues below by λ, we bound amplification factors:

$$\text{Amplification} = \frac{1}{\mu_j + \lambda} \leq \frac{1}{\lambda} \tag{17}$$

Comparison table:

λ	Min Eigenvalue	Condition Number	Max Amplification	Stability
0 (OLS)	0.001	100,000	1000×	Unstable
0.01	0.011	9,091	91×	Still problematic
1	1.001	101	1×	Acceptable
10	10.001	11	0.1×	Excellent

The price: Bias is introduced (solution shifts away from OLS). But this is a principled tradeoff:

Why Model Selection is Ill-Posed

Model selection faces the same issue at a higher level.

Example: The Model Selection Problem

Given:

- Training data: $(x_1, y_1), ..., (x_n, y_n)$
- Model class: $\mathcal{M} = \{\text{linear}, \text{degree-2}, ..., \text{degree-15}\}$

Goal: Select $M^* \in \mathcal{M}$ that minimizes test error.

Why it's ill-posed:

1. **Non-existence:** May not have a single "best" model (multiple models tie)
2. **Non-uniqueness:** Different selection criteria give different answers
3. **Instability:** Small data changes → completely different model selected

Criterion 3 is the killer: bootstrap resampling often gives different degree polynomials!

Regularization stabilizes model selection:

Instead of discrete choice (degree 1, 2, ..., 15), use continuous parameter λ:

Small changes in data → small changes in optimal λ → continuous adjustment rather than discrete jumps.

Summary: From Ill-Posed to Well-Posed

🎯 **Key Takeaway**

Regularization transforms ill-posed inverse problems into well-posed ones:

Before (Ill-Posed):

- Small eigenvalues → unstable inversion
- Noise amplified by factor 1/μ_min
- Solution technically exists but practically unusable
- Hadamard criterion 3 violated (instability)

After (Well-Posed):

- Eigenvalues bounded below by λ
- Noise amplification bounded by 1/λ
- Solution stable and usable
- All three Hadamard criteria satisfied

The tradeoff: Accept small bias (shift from OLS) for massive stability gain.

Progress on backbone: We've seen that the instability in OLS comes from small eigenvalues in $X^\top X$. Ridge regression fixes this by adding λI, transforming ill-posed → well-posed. But how exactly does ridge shrink coefficients? The answer lies in understanding the shrinkage mechanism.

Tikhonov Regularization Framework

Motivation: We need a systematic way to trade bias for variance. Tikhonov regularization does this by adding a penalty term to the objective: instead of minimizing data misfit alone (which is ill-posed), we minimize data misfit plus a measure of solution complexity. The regularization parameter α controls the tradeoff—it's the dial that moves us along the bias-variance seesaw toward stability.

The Fundamental Question: How Do We Control Model Complexity?

? Student Question

"Adding $\lambda\|\beta\|^2$ to the objective makes it LARGER. We're literally making the optimization problem harder by preventing ourselves from minimizing training error. How can adding a penalty that increases the objective value possibly help?"

This is a profound question that cuts to the heart of regularization. Let's investigate systematically.

The Natural Approach: Minimize Training Error Alone

The most obvious approach is to fit the data as well as possible:

$$\hat{\beta} = \arg\min_{\beta}\|y - X\beta\|^2 \tag{18}$$

This is clean, principled, and unbiased. Let's try it on a challenging problem.

Example: Gene Expression Prediction

We want to predict disease outcome from gene expression levels:

- n = 100 patients
- d = 50 genes measured for each patient
- Goal: Predict binary outcome (disease/healthy)

Each gene is a potential biomarker. We don't know which ones matter, so we include all 50.

OLS solution with all 50 genes:

The model memorized training noise rather than learning genuine patterns.

Why Not Feature Selection?

"If 50 genes is too many, just select a subset!"

This seems reasonable. Let's try:

Forward selection approach:

1. Start with 0 features → Test error: 0.50 (predicting mean)
2. Add best feature → Test error: 0.38
3. Add 2nd best → Test error: 0.31
4. Continue adding...
5. At 15 features → Test error: 0.23 (best!)
6. At 20 features → Test error: 0.29 (getting worse, stop!)

Result: Use 15 genes, discard 35 genes.

But wait—multiple problems:

⚠️ **Crisis**

The Combinatorial Explosion

With d = 50 features, there are $2^{50} \approx 1.13 \times 10^{15}$ possible subsets to consider.

At 1 million evaluations per second: $\frac{10^{15}}{10^6 \cdot 86400} \approx 35800$ years!

Even the greedy forward selection is unstable—different cross-validation splits select completely different genes.

Issue	Impact	Example	Consequence
Combinatorial	2^d subsets	2^50 ≈ 10^15	Computationally intractable
Discontinuous	Add/remove causes jumps	14 genes: 0.25 15 genes: 0.23 16 genes: 0.27	Can't interpolate
Greedy suboptimal	Locally optimal choices	Genes 1,3,7 selected But 2,4,8 might be better	Miss optimal combination
Unstable	Different splits → different features	Split A: genes {1,3,5,7,9} Split B: genes {2,4,6,8,10}	No consistent signature
Information loss	Discarded features had signal	35 genes thrown away Lost their contributions	Reduced predictive power

Comparison of feature counts:

Method	Features Used	Test Error	Problem
OLS (all 50)	50	0.82	Overfitting, unstable
Forward (15)	15	0.23	Lost signal from 35 genes
Backward (18)	18	0.25	Different choice, no principled criterion
Exhaustive search	? (optimal)	?	Would take 35,800 years!

The fundamental issue: Feature selection forces **binary decisions** (in or out). We need **continuous control** over how much each feature contributes.

The Attempted Compromise: Manual Tuning

"What if we just add a tiny constant like $0.0001 \cdot I$ to stabilize numerics?"

$$\hat{\beta} = (X^\top X + 0.0001 \cdot I)^{-1} X^\top y \tag{19}$$

Result:

Why it failed: The constant 0.0001 is arbitrary and doesn't address the root problem. We're not trying to avoid numerical errors—we're trying to control the bias-variance tradeoff.

We need a **principled** penalty, not a numerical hack.

The Resolution: Continuous Regularization

What if instead of discrete feature selection, we use a **continuous penalty** that smoothly controls all coefficients?

> 💡 **Aha Moment**
>
> **The Key Insight: Regularization as a Continuous Dial**
>
> Add penalty $\lambda\|\beta\|^2$ where λ is tuned to balance bias and variance:
>
> $$\hat{\beta}_\lambda = \arg\min_\beta [\|y - X\beta\|^2 + \lambda\|\beta\|^2] \tag{20}$$
>
> As we vary λ from 0 to ∞:
> - $\lambda = 0$: All features fully used (high variance)
> - λ small: Most features contribute (controlled variance)
> - λ optimal: Important features dominate (balanced)
> - λ large: All features heavily shrunk (high bias)
> - $\lambda \to \infty$: All coefficients $\to 0$ (predict mean)
>
> Transform: Discrete search (2^d possibilities) $\to$ 1D optimization (find optimal λ)

Ridge solutions at different λ values:

λ	"Effective Features"	$\|\hat{\beta}_\lambda\|$	Test Error	Interpretation
0	50 (all fully)	847	0.82	Unregularized OLS, variance explosion
0.01	45	412	0.62	Light regularization, still unstable
0.1	35	185	0.31	Moderate, good progress
1.0	15	58	0.23	**Optimal: balanced bias-variance**
10	5	12	0.29	Too much bias, underfitting
100	1	2.1	0.42	Severe bias, almost predicting mean
∞	0	0	0.50	Predicting mean exactly

The "effective features" is measured by degrees of freedom: $\text{df}(\lambda) = \sum_{j=1}^{d} \frac{\mu_j}{\mu_j + \lambda}$, which quantifies how many features effectively contribute.

The breakthrough:

Visualizing the Regularization Path

Andrey Tikhonov (1963) developed this framework for solving ill-posed problems in geophysics and inverse scattering. The same ideas were independently discovered in statistics as "ridge regression" (Hoerl & Kennard, 1970).

As λ increases, coefficients shrink continuously:

Path for 5 most important genes:

λ	Gene 1	Gene 2	Gene 3	Gene 4	Gene 5
0	42.5	−38.2	31.7	−28.3	25.1
0.1	28.3	−25.1	21.4	−18.9	16.8
1.0	8.7	−7.5	6.9	−5.8	5.2
10	1.2	−1.0	0.9	−0.8	0.7
100	0.1	−0.1	0.1	−0.1	0.1

Notice:

Why This Resolves the Crisis

🎯 Key Takeaway

Regularization transforms an intractable discrete optimization into a tractable continuous one:

Before (Feature Selection):

- Search space: 2^d discrete subsets
- Optimization: Combinatorial (NP-hard)
- Stability: Unstable (different folds → different features)
- Information: Binary (in or out)

After (Regularization):

- Search space: $\lambda \in [0, \infty)$ continuous
- Optimization: 1D line search or cross-validation
- Stability: Smooth (small λ change → small β change)
- Information: Soft weighting (all features contribute proportionally)

The penalty term $\lambda\|\beta\|^2$ is not making optimization harder—it's making generalization **better.**

Yes, training error increases (0.05 → 0.24). But test error plummets (0.82 → 0.23). The small bias cost buys massive variance reduction!

General Formulation

Now that we understand WHY we need regularization, let's see the general framework.

Definition: Tikhonov Regularization

Let $\mathcal{H}$ be a **Hilbert space** (a complete vector space with an inner product, introduced in Week 1), $\Pi : \mathcal{A} \subseteq \mathcal{H} \to \mathbb{R}^d$ a (possibly nonlinear) **operator** (a function mapping between function spaces), and π^δ noisy observations with $\|\pi - \pi^\delta\| \leq \delta$.

A standard Tikhonov formal template is:

$$a_\alpha^\delta = \arg\min_{a \in \mathcal{A}} \left[\|\Pi(a) - \pi^\delta\|^2 + \alpha \|a - a_0\|_{\mathcal{H}}^2 \right] \tag{21}$$

where:

- $\alpha > 0$: **regularization parameter** (controls bias-variance tradeoff)
- $a_0 \in \mathcal{H}$: **prior** or **initial guess** (our belief about the solution before seeing data)

Existence of a minimizer requires additional closedness/compactness assumptions. In this sheet, the template mainly motivates finite-dimensional ridge regression.

The objective has two competing terms:

1. **Data fidelity** $\|\Pi(a) - \pi^\delta\|^2$: measures how well the solution fits observed data (low value = good fit)
2. **Regularization** $\alpha\|a - a_0\|^2$: penalizes solutions far from the prior a_0, enforcing simplicity/smoothness (low value = simple model)

Example: Housing Prices - Tikhonov Trust Region

Think of Tikhonov as a "trust region" approach: we trust the data (first term) but not completely—we also trust a prior belief a_0 (second term). The parameter α sets how much we trust the prior vs. the data.

- α small: "I trust my data"—fit closely, risk instability
- α large: "I trust my prior"—ignore data, high bias
- α optimal: balanced trust—best generalization

🤔 **Pause and Think**

Question: What happens as $\alpha \to 0$? As $\alpha \to \infty$?

Hint: Consider the two terms in the objective function and which dominates in each limit.

Answer: $\alpha \to 0$: Unregularized solution (ill-posed). $\alpha \to \infty$: $a_\alpha^\delta \to a_0$ (ignore data completely).

Theoretical Guarantees

The next statements are qualitative inverse-problem facts. Their full versions require technical compactness, closedness, and source-condition assumptions; ridge regression below is the finite-dimensional case we will actually compute.

Theorem: Stability (Schematic)

Under standard compactness and continuity assumptions, for fixed $\alpha > 0$, Tikhonov minimizers depend continuously on the observed data.

Interpretation: Small data perturbations lead to small changes in the regularized solution.

Theorem: Convergence (Schematic)

If the noise level tends to zero and the regularization parameter α also tends to zero at a compatible rate, Tikhonov solutions approach an appropriate exact solution, often the solution closest to the prior a_0.

Theorem: Convergence Rate (Schematic)

Under additional smoothness/source conditions, one can prove quantitative convergence rates. The precise rate depends on the source condition and the choice of α.

🎯 **Key Takeaway**

Tikhonov regularization transforms an ill-posed problem into a well-posed one. The price is bias (solution differs from unregularized), but the gain is stability (continuous dependence on data).

Choosing the Regularization Parameter

Now that we have the framework, how do we choose λ (or α)?

Definition: Discrepancy Principle (Morozov)

Select the largest α such that:

$$\left\|\Pi\left(a_\alpha^\delta\right) - \pi^\delta\right\| \leq \tau\delta \tag{22}$$

where $\tau > 1$ is a safety factor (typically $\tau \in [1.1, 2]$).

Intuition: Don't fit data better than the noise level allows.

🤔 **Pause and Think**

Question: Why choose the largest α satisfying the constraint, not the smallest?

Hint: Think about what larger α means for solution stability.

Answer: Larger α means more regularization → more stable solution. We want maximum stability subject to fitting data within noise level.

Practical approaches when noise level is unknown:

1. **Cross-validation** (most common):
   - Split data into K folds
   - For each λ: train on K-1 folds, test on held-out fold
   - Choose λ minimizing average test error
   - Computationally expensive but robust
2. **Leave-One-Out Cross-Validation (LOOCV)**:

? Student Question

"Computing LOO-CV requires refitting the model n times—once for each held-out point. For $n = 1000$ and $d = 50$, this means 1000 matrix inversions. For large datasets, this seems prohibitively expensive. Is there a better way?"

⚠ Crisis

The Naive Approach:

For each $i = 1, ..., n$:

1. Remove (x_i, y_i) from the training set
2. Fit $\hat{\beta}_\lambda^{(-i)}$ on the remaining $n - 1$ points
3. Compute squared prediction error: $\left(y_i - x_i^\top \hat{\beta}_\lambda^{(-i)}\right)^2$
4. Average over all i

Computational cost: $O(n \cdot d^3)$ — need to invert $(X^\top X + \lambda I)$ a total of n times!

Example: For $n = 1000, d = 50$: 1000 inversions of 50×50 matrices $\approx$ 125 million operations

💡 **Aha Moment**

The Sherman-Morrison Resolution

The hat matrix diagonal gives LOO predictions **without refitting**:

$$\text{LOOCV}(\lambda) = \frac{1}{n}\sum_{i=1}^{n}\left[\frac{y_i - \hat{y}_i}{1 - H_{ii}}\right]^2 \tag{23}$$

where:

- $\hat{y}_i = x_i^\top \hat{\beta}_\lambda$ is the prediction using **all** n points
- $H_{ii} = x_i^\top (X^\top X + \lambda I)^{-1} x_i$ is the **leverage** of point i

Why this works: The Sherman-Morrison formula shows that removing point i is a rank-1 update to $X^\top X$, and the LOO prediction can be recovered from the full-data prediction via:

$$y_i - x_i^\top \hat{\beta}_\lambda^{(-i)} = \frac{y_i - \hat{y}_i}{1 - H_{ii}} \tag{24}$$

Computational cost: $O(d^3)$ for one matrix inversion + $O(nd^2)$ for computing all leverages H_{ii}

Transformation: n refits $\to$ 1 refit + lightweight diagonal extraction!

Derivation via Sherman-Morrison Formula:

The key insight is that removing point i updates $X^\top X$ by a rank-1 perturbation. Let X_{-i} denote the design matrix with row i removed. Then:

$$X^\top X = X_{-i}^\top X_{-i} + x_i x_i^\top \tag{25}$$

The Sherman-Morrison formula for rank-1 updates states:

$$(A + uv^\top)^{-1} = A^{-1} - \frac{A^{-1}uv^\top A^{-1}}{1 + v^\top A^{-1}u} \tag{26}$$

Applying this to relate $(X_{-i}^\top X_{-i} + \lambda I)^{-1}$ to $(X^\top X + \lambda I)^{-1}$, and performing algebraic manipulation, we obtain:

$$\hat{\beta}_\lambda^{(-i)} = \hat{\beta}_\lambda - \frac{(X^\top X + \lambda I)^{-1} x_i \left(y_i - x_i^\top \hat{\beta}_\lambda\right)}{1 - H_{ii}} \tag{27}$$

Multiplying both sides by $x_i^\top$ and rearranging:

$$x_i^\top \hat{\beta}_\lambda^{(-i)} = x_i^\top \hat{\beta}_\lambda - \frac{H_{ii}\left(y_i - x_i^\top \hat{\beta}_\lambda\right)}{1 - H_{ii}} \tag{28}$$

Therefore:

$$y_i - x_i^\top \hat{\beta}_\lambda^{(-i)} = y_i - x_i^\top \hat{\beta}_\lambda + \frac{H_{ii}\left(y_i - x_i^\top \hat{\beta}_\lambda\right)}{1 - H_{ii}} = \frac{y_i - x_i^\top \hat{\beta}_\lambda}{1 - H_{ii}} \tag{29}$$

This is the LOO residual expressed using only the full-data fit!

🎯 Key Takeaway

Interpreting Leverage:

The leverage H_{ii} measures how much point i influences its own prediction:

- **High leverage** ($H_{ii} \approx 1$): Point i has strong self-influence → removing it changes the fit significantly → LOO error is **amplified** by factor $\frac{1}{1-H_{ii}}$
- **Low leverage** ($H_{ii} \approx 0$): Point i is typical → removing it barely changes the fit → LOO error ≈ ordinary residual

The factor $\frac{1}{1-H_{ii}}$ corrects for the fact that the full-data prediction $\hat{y}_i$ is "contaminated" by point i itself. High-leverage points contaminate their own predictions more, so we need a larger correction.

3. **Generalized Cross-Validation (GCV)**:

   GCV approximates LOOCV by replacing individual leverages H_{ii} with the **average leverage**:

   $\text{GCV}(\lambda) = \frac{n\left\|y - X\hat{\beta}_\lambda\right\|^2}{(n - \text{df}(\lambda))^2}$

   where $\text{df}(\lambda) = \text{trace}(H) = \sum_{i=1}^{n} H_{ii}$ is the sum of all leverages (effective degrees of freedom).

   Connection to LOOCV: If all leverages were equal ($H_{ii} = \frac{\text{df}(\lambda)}{n}$ for all i), then GCV would equal LOOCV exactly. GCV is faster to compute (only need the trace, not all diagonal elements) but less accurate when leverage varies significantly across points.

4. **AIC/BIC** (information criteria):
   - Balance fit quality and model complexity
   - $\text{AIC} = n \log\left(\frac{\text{RSS}}{n}\right) + 2 \cdot \text{df}(\lambda)$
   - Asymptotic approximations to out-of-sample error

→ Week 5 Preview: Cross-Validation

The discrepancy principle requires knowing the noise level δ, which we rarely know in practice. Next week, we'll learn cross-validation—a data-driven method to choose α (or λ) without knowing δ or using test data.

🔨 Construct Your Own

Explore Regularization Strength

Consider the regularized objective $\left\|\Pi(a) - \pi^\delta\right\|^2 + \alpha\|a\|^2$.

1. For $\alpha \in \{0.01, 0.1, 1, 10, 100\}$, sketch qualitatively how the solution a_α changes
2. Identify which α values lead to: (a) overfitting, (b) underfitting, (c) good generalization
3. How would you visualize the "regularization path" (solution vs. α)?

Challenge: Can you prove that $\|a_\alpha\|$ is monotonically decreasing in α?

Progress on backbone: The Tikhonov framework shows us **how** to regularize—add a penalty term scaled by α, providing continuous control over model complexity. We've seen this resolves the combinatorial explosion of feature selection. Next, we'll see the most important special case: ridge regression for linear models.

Ridge Regression

Motivation: Ridge regression is Tikhonov regularization applied to linear least squares. It's the simplest nontrivial regularized method, yet it's used everywhere—from genomics to finance to machine learning. Why? Because it has a closed-form solution, clear geometric interpretation, and serves as a building block for more sophisticated methods (elastic net, generalized ridge, etc.). Master ridge, and you've mastered the core ideas of regularization.

Why Does OLS Fail? A Concrete Crisis

? Student Question

"OLS is the Best Linear Unbiased Estimator (BLUE). If we shrink the coefficients toward zero with ridge regression, won't we make predictions worse by introducing bias?"

This is a natural and important question. Let's investigate by trying the "obvious" approach first: use OLS.

Problem Setup

Consider linear regression: $y = X\beta + \varepsilon$ where:

Definition: Ordinary Least Squares (OLS)

Ordinary Least Squares (OLS) finds parameters that minimize squared prediction error on training data. If $X^\top X$ is invertible, the OLS estimator is:

$$\hat{\beta}_{\text{OLS}} = \arg\min_\beta \|y - X\beta\|^2 = (X^\top X)^{-1} X^\top y. \tag{30}$$

If $X^\top X$ is singular (not invertible), minimizers may not be unique and the **Moore-Penrose pseudoinverse** (a generalization of matrix inverse) gives the minimum-norm OLS solution.

This looks clean and elegant. OLS is unbiased, has a closed form, and is taught in every statistics course. What could go wrong?

The Natural Approach: Apply OLS

Let's apply OLS to a real problem: predicting housing prices from features that naturally correlate.

Example: Housing Price Prediction

We have n = 100 houses with d = 5 features:

- x_1: Square footage (mean ≈ 2000 sq ft)
- x_2: Number of bedrooms (mean ≈ 3)
- x_3: Number of bathrooms (mean ≈ 2)
- x_4: Lot size in sq ft (mean ≈ 5000)
- x_5: Age in years (mean ≈ 20)

These features are naturally correlated:

- Bigger houses tend to have more bedrooms and bathrooms
- More bedrooms usually means more square footage
- Larger lots often accompany larger houses

We fit OLS using the formula $\hat{\beta}_{\text{OLS}} = (X^\top X)^{-1} X^\top y$.

The results:

$$\hat{\beta}_{\text{OLS}} = \begin{pmatrix} 500 \\ -30000 \\ 25000 \\ -8 \\ 100 \end{pmatrix} \tag{31}$$

Wait… let's interpret these coefficients:

Performance:

The Multicollinearity Trap

⚠️ **Crisis**

The Fatal Flaw: Noise Amplification

When features are correlated (multicollinearity), $X^\top X$ has small eigenvalues. These small eigenvalues act as dividers in $(X^\top X)^{-1}$, causing tiny noise in y to be **amplified** into huge swings in $\hat{\beta}_{\text{OLS}}$.

The coefficients aren't "true"—they're noise-amplified nonsense!

Let's dig deeper into why this happened:

Eigenvalue analysis of $X^\top X$:

$$\text{Eigenvalues: } \mu_1 = 15000, \mu_2 = 3200, \mu_3 = 450, \mu_4 = 12, \mu_5 = 0.8 \tag{32}$$

The smallest eigenvalue $\mu_5 = 0.8$ is tiny compared to the largest $\mu_1 = 15000$.

Condition number:

$$\kappa = \frac{\mu_{\max}}{\mu_{\min}} = \frac{15000}{0.8} \approx 18750 \tag{33}$$

This means: the precision of the solution is degraded by nearly 19,000×! Even small measurement noise gets amplified catastrophically.

Aspect	Value	Impact	Consequence
Smallest eigenvalue	$\mu_5 = 0.8$	Divider in inverse	Amplifies noise 1250×
Condition number	$\kappa \approx 18750$	Precision loss	4-5 digits lost
Coefficient signs	2 wrong	Nonsensical	Can't interpret model
Test error	0.68	4.5× training	Severe overfitting

What happened geometrically:

When features are nearly collinear (e.g., $x_1 \approx 2x_2 - x_3$), there are many coefficient combinations that fit the training data equally well:

OLS picks one arbitrarily based on tiny numerical noise. Different training samples give wildly different coefficients—hence the catastrophic test error.

Attempted Fix 1: Remove Correlated Features

"If multicollinearity is the problem, just remove redundant features!"

Let's drop x_2 (bedrooms) since it's correlated with x_1 (square footage).

Result with 4 features:

$$\hat{\beta}_{\text{OLS}} = \begin{pmatrix} 600 \\ (\text{dropped}) \\ 18000 \\ 5 \\ 120 \end{pmatrix} \tag{34}$$

Issues:

1. Still has interpretation problems ($\beta_3 = 18000$ for bathrooms?!)
2. Lost information from bedrooms (they DO predict price!)
3. Which feature to drop? Arbitrary choice
4. Test MSE: 0.51 (better, but still 3.4× training error)

Method	Features Used	Test MSE	Problem
OLS (all 5)	5	0.68	Nonsensical coefficients
OLS (drop bedrooms)	4	0.51	Lost signal, still unstable
OLS (drop bathrooms)	4	0.49	Different choice, different result

The fundamental problem: Feature selection is a **discrete** decision. Removing one feature causes discontinuous jumps. We need **continuous** control.

Attempted Fix 2: Add Tiny Constant (Numerical Hack)

"Just add $0.0001 \cdot \text{I}$ to avoid division by tiny numbers!"

This is sometimes done for numerical stability. Let's try:

$$\hat{\beta} = (X^\top X + 0.0001 \cdot I)^{-1} X^\top y = \begin{pmatrix} 498 \\ -29800 \\ 24900 \\ -7.8 \\ 99 \end{pmatrix} \tag{35}$$

Result: Barely changed! The problem isn't numerical precision—it's the fundamental ill-posedness of the problem.

The Resolution: Ridge Regression

What if, instead of arbitrarily removing features or adding a tiny constant, we **systematically** add a penalty that stabilizes all directions proportional to how unstable they are?

💡 **Aha Moment**

The Key Insight

Add λI to $X^\top X$ where λ is chosen to balance bias and variance, not just avoid numerical errors.

This transforms:

- Unstable directions (small eigenvalues) $\rightarrow$ heavily stabilized
- Stable directions (large eigenvalues) $\rightarrow$ lightly touched
- All coefficients $\rightarrow$ shrunk toward zero by data-driven amounts

Ridge Regression Solution

Definition: Ridge Regression

Ridge regression adds an ℓ^2 penalty (sum of squared coefficients) to ordinary least squares:

$$\hat{\beta}_\lambda = \arg\min_\beta [\|y - X\beta\|^2 + \lambda \|\beta\|^2] \tag{36}$$

where $\lambda > 0$ is the **regularization parameter** (controls the strength of the penalty; larger λ = more regularization = smaller coefficients).

The two terms represent competing goals: fit the data well (first term) vs. keep coefficients small (second term).

Theorem: Closed-Form Solution

For $\lambda > 0$, the ridge estimator is:

$$\hat{\beta}_\lambda = (X^\top X + \lambda I)^{-1} X^\top y \tag{37}$$

Proof. The objective is:

$$J(\beta) = \|y - X\beta\|^2 + \lambda\|\beta\|^2 = (y - X\beta)^\top(y - X\beta) + \lambda\beta^\top\beta \tag{38}$$

Taking the gradient with respect to β (using the matrix calculus identities $\nabla_\beta(a^\top\beta) = a$ and $\nabla_\beta(\beta^\top A\beta) = 2A\beta$ for symmetric A):

$$\nabla_\beta J = -2X^\top y + 2X^\top X\beta + 2\lambda\beta = 0 \tag{39}$$

Solving for β by setting the gradient to zero (this is a necessary condition for a minimum):

$$(X^\top X + \lambda I)\beta = X^\top y \implies \beta = (X^\top X + \lambda I)^{-1}X^\top y \tag{40}$$

The **Hessian** (matrix of second derivatives) is $\nabla^2 J = 2(X^\top X + \lambda I) \succ 0$ (positive definite for $\lambda > 0$), confirming this is a global minimum. ■

Applying Ridge to Housing Data

Let's apply ridge regression with $\lambda = 10$ to our housing price problem.

Ridge solution ($\lambda = 10$):

$$\hat{\beta}_{\text{ridge}} = \begin{pmatrix} 150 \\ 5000 \\ 8000 \\ 2 \\ -500 \end{pmatrix} \tag{41}$$

Interpretation:

All signs are correct! All magnitudes are sensible!

Performance:

The comparison:

Method	Training MSE	Test MSE	Coefficient Signs	Verdict
OLS	0.15	0.68	2/5 wrong	Nonsensical, unstable
Feature selection	0.18	0.51	Still issues	Lost information
Ridge ($\lambda = 10$)	0.24	0.18	5/5 correct	Sensible, stable ✓

The gap: Ridge achieves 3.8× lower test error than OLS, despite having higher training error. The small increase in bias (training error) is vastly outweighed by the massive reduction in variance!

How Ridge Fixes the Problem

Ridge modifies the eigenvalues of $X^\top X$:

Original eigenvalues:

$$\mu_1 = 15000, \mu_2 = 3200, \mu_3 = 450, \mu_4 = 12, \mu_5 = 0.8 \tag{42}$$

Modified eigenvalues ($X^\top X + \lambda I$ with $\lambda = 10$):

$$\mu_1 + 10 = 15010, \mu_2 + 10 = 3210, \mu_3 + 10 = 460, \mu_4 + 10 = 22, \mu_5 + 10 = 10.8 \tag{43}$$

Shrinkage factors (how much each direction is preserved):

Direction	Original μ_j	Modified $\mu_j + \lambda$	Shrinkage Factor	Effect
1 (most stable)	15000	15010	0.999	Barely shrunk
2	3200	3210	0.997	Barely shrunk
3	450	460	0.978	Lightly shrunk
4	12	22	0.545	Moderately shrunk
5 (most unstable)	0.8	10.8	0.074	**Heavily shrunk** ← key!

The shrinkage factor for direction j is $\frac{\mu_j}{\mu_j+\lambda}$.

The key insight: Ridge automatically shrinks coefficients proportionally to how unstable they are!

Direction 5 (the most unstable, with $\mu_5 = 0.8$) gets shrunk by 92.6%! This is exactly where OLS was amplifying noise catastrophically.

Why Ridge Works Better Than Feature Selection

Aspect	Feature Selection	Ridge Regression
Decision type	Discrete (in/out)	Continuous (shrinkage factors)
Stability	Unstable (different folds → different features)	Stable (smooth in data)
Information use	Throws away features completely	Uses all features, weighted by stability
Computation	Combinatorial (2^d subsets)	Single matrix inverse
Tuning	Which features to keep?	One parameter λ

Comparison with 50 gene expression features:

Method	Effective Features	Test MSE	Computational Cost
All features (OLS)	50	0.82	$O(d^3)$
Forward selection	15	0.41	$O(d^3 \cdot k)$ iterations
Ridge ($\lambda = 1$)	15 (effective)	0.23	$O(d^3)$

Ridge achieves the best test error while using information from **all** features through soft weighting!

Key Properties

🎯 Key Takeaway

Ridge regression resolves the multicollinearity crisis through **selective shrinkage**:

1. **Always well-defined:** $X^\top X + \lambda I$ is invertible for any $\lambda > 0$, even when $d > n$
2. **Selective shrinkage:** Shrinks unstable directions (small eigenvalues) more than stable directions
3. **Bias-variance tradeoff:** Accepts small bias (shrinkage) for massive variance reduction
4. **Continuous control:** λ provides a dial from high-variance ($\lambda \to 0$) to high-bias ($\lambda \to \infty$)
5. **Interpretable:** Coefficients have sensible signs and magnitudes

✓ Checkpoint

Question: Why does adding λI to $X^\top X$ guarantee invertibility?

Answer: Because λI shifts all eigenvalues by λ. If $X^\top X$ has eigenvalues $\mu_1, ..., \mu_d \geq 0$, then $X^\top X + \lambda I$ has eigenvalues $\mu_1 + \lambda, ..., \mu_d + \lambda > 0$ (all positive, hence invertible).

🤔 Pause and Think

Question: What happens to $\hat{\beta}_\lambda$ as $\lambda \to 0$? As $\lambda \to \infty$?

Hint: Consider the limiting behavior of $(X^\top X + \lambda I)^{-1}$ in each case.

Answer: $\lambda \to 0$: $\hat{\beta}_\lambda \to \hat{\beta}_{\text{OLS}}$ (unregularized). $\lambda \to \infty$: $\hat{\beta}_\lambda \to 0$ (all coefficients shrink to zero).

Remark

For interested students: The formula $\hat{\beta}_\lambda = (X^\top X + \lambda I)^{-1} X^\top y$ extends beautifully to infinite-dimensional spaces through **kernel ridge regression**. The "kernel trick" allows ridge regression in reproducing kernel Hilbert spaces (RKHS) without ever computing in the high-dimensional space explicitly. This is an advanced topic not covered in this course but important in modern machine learning.

Summary and Looking Ahead

🎯 Key Takeaway

Main Insights from Week 4:

1. **Bias-variance tradeoff:** Model complexity creates tension between systematic error (bias) and estimation instability (variance).
2. **Regularization as stabilization:** Tikhonov regularization transforms ill-posed inverse problems into well-posed ones by adding a penalty term.
3. **Ridge regression:** Closed-form solution $(X^\top X + \lambda I)^{-1} X^\top y$ shrinks coefficients, especially in directions with small eigenvalues.

Next week: How do we choose λ in practice? We will study cross-validation, information criteria (AIC/BIC), and other model selection techniques.

Connection to future topics:

→ Week 5 Preview: Model Selection via Cross-Validation

We've derived ridge and understood its bias-variance tradeoff, but how do we choose λ without a validation set? Next week's cross-validation provides an elegant answer: partition the training data into folds, use each fold as a "mini test set" for the others, and select the λ that minimizes average fold error. This is the gold standard for tuning regularization parameters.

Remark

For interested students: Ridge regression extends to infinite dimensions via the kernel trick. The dual formulation $\hat{\alpha} = (K + \lambda I)^{-1} y$ (where K is the kernel matrix) allows ridge in reproducing kernel Hilbert spaces (RKHS) without ever computing the feature map explicitly. This powers support vector machines and Gaussian process regression, though these topics are beyond the scope of this course.

Additionally, there is an **Ivanov (constrained) formulation** of ridge regression: minimize $\|y - X\beta\|^2$ subject to $\|\beta\|^2 \leq t$. This is equivalent to the Tikhonov (penalized) formulation via Lagrange duality—for each $\lambda > 0$, there exists a constraint $t > 0$ giving the same solution.

🧩 **Synthesis Challenge**

Grand Challenge: Unifying Regularization and Generalization

Synthesize Weeks 2-4 by answering:

1. How does VC dimension relate to the bias-variance decomposition?
2. How does regularization effectively reduce VC dimension?
3. Why does the PAC bound improve with regularization?
4. Can you express the optimal λ in terms of n, d, σ^2, and properties of β^*?

Hint: Think of regularization as restricting the hypothesis class from $\mathcal{H}$ to $\mathcal{H}_\lambda = \{h : \|h\| \leq R(\lambda)\}$, where $R(\lambda)$ decreases with λ.

🧠 **Mental Model: The Week 4 Conceptual Map**

```
Bias-Variance Decomposition
        ↓
Complex models → High variance (instability)
        ↓
Ill-Posed Inverse Problems
        ↓
Tikhonov Regularization (add penalty)
        ↓
Ridge Regression (quadratic penalty)
        ↓
Shrinkage toward zero
        ↓
Reduced variance, increased bias
        ↓
Optimal λ balances bias-variance tradeoff
```

Every concept builds on the last: we diagnose the problem (bias-variance), identify its mathematical nature (ill-posedness), apply a general solution framework (Tikhonov), specialize to the most important case (ridge), and understand its effect (shrinkage). This logical flow is the backbone of Week 4.

Student Challenges

Student Challenge

1. Prove that ridge regression with $\lambda \to \infty$ converges to $\hat{\beta} = 0$.
2. Implement ridge regression and visualize the regularization path for a dataset with multicollinearity.
3. Derive the bias-variance decomposition for classification with 0-1 loss (harder than regression case).
4. Show that ridge is the MAP estimator under Bayesian prior $\beta \sim \mathcal{N}\left(0, \frac{\sigma^2}{\lambda} I\right)$.

Proof Template: Proving Optimality of Ridge via Second Derivative Test

Goal: Show that $\hat{\beta}_\lambda = (X^\top X + \lambda I)^{-1} X^\top y$ minimizes $J(\beta) = \|y - X\beta\|^2 + \lambda\|\beta\|^2$.

Step 1: Compute first derivative $\nabla_\beta J$ and verify it vanishes at $\hat{\beta}_\lambda$.

Step 2: Compute second derivative (Hessian) $\nabla^2 J = \ldots$ (fill in).

Step 3: Show the Hessian is positive definite for $\lambda > 0$ by checking eigenvalues.

Step 4: Conclude that $\hat{\beta}_\lambda$ is a global minimum (convexity ensures local = global).

Work through each step with explicit matrix computations.

Proof Template: Deriving Variance of Ridge Estimator

Given: $\hat{\beta}_\lambda = (X^\top X + \lambda I)^{-1} X^\top y$ and $y = X\beta^* + \varepsilon$ with $\mathrm{Cov}(\varepsilon) = \sigma^2 I$.

Goal: Show $\mathrm{Cov}\left(\hat{\beta}_\lambda\right) = \sigma^2 (X^\top X + \lambda I)^{-1} X^\top X (X^\top X + \lambda I)^{-1}$.

Step 1: Write $\hat{\beta}_\lambda = (X^\top X + \lambda I)^{-1} X^\top (X\beta^* + \varepsilon)$.

Step 2: Note that variance comes only from the ε term (why?).

Step 3: Use $\mathrm{Cov}(A\varepsilon) = A\,\mathrm{Cov}(\varepsilon) A^\top$ with $A = (X^\top X + \lambda I)^{-1} X^\top$.

Step 4: Simplify: $A\,\mathrm{Cov}(\varepsilon) A^\top = A(\sigma^2 I) A^\top = \sigma^2 A A^\top = \ldots$.

Complete the calculation.

Theorem Landscape

Week 4 Theorem Hierarchy

Foundation:

- Bias-Variance Decomposition: Every estimator's MSE splits into bias^2 + variance + irreducible error

Regularization Theory (Schematic):

- Stability: Regularized solutions depend continuously on data
- Convergence: As noise $\to 0$ and regularization parameter $\to 0$ appropriately, solutions approach exact solution
- Convergence Rate: Quantitative convergence rates under smoothness conditions

Application to Ridge:

- Closed-Form Solution: $\hat{\beta}_\lambda = (X^\top X + \lambda I)^{-1} X^\top y$

Key Properties (derived in text):

- Shrinkage: Ridge shrinks each principal component by factor $\frac{\mu_j}{\mu_j + \lambda}$
- Bias-Variance Tradeoff: Small bias increase for large variance reduction

These results form a complete story: decompose error $\to$ understand ill-posedness $\to$ apply regularization $\to$ analyze bias-variance.